

Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing

Xi Victoria Lin Richard Socher Caiming Xiong
Salesforce Research
{xilin,rsocher,cxiong}@salesforce.com

This work first appeared in Findings of EMNLP 2020. Here we report the performance and analysis of the BRIDGE model using BERT-large, on both the Spider and WikiSQL datasets. We also extend the discussion on model variance and present an ensemble model that significantly outperforms single models on Spider.

Abstract

We present BRIDGE, a powerful sequential architecture for modeling dependencies between natural language questions and relational databases in cross-DB semantic parsing. BRIDGE represents the question and DB schema in a tagged sequence where a subset of the fields are augmented with cell values mentioned in the question. **The hybrid sequence is encoded by BERT with minimal subsequent layers and the text-DB contextualization is realized via the fine-tuned deep attention in BERT.** Combined with a pointer-generator decoder with schema-consistency driven search space pruning, BRIDGE attained state-of-the-art performance on popular cross-DB text-to-SQL benchmarks, Spider (71.1% dev, 67.5% test with ensemble model) and WikiSQL (92.6% dev, 91.9% test). Our analysis shows that BRIDGE effectively captures the desired cross-modal dependencies and has the potential to generalize to more text-DB related tasks. Our implementation is available at <https://github.com/salesforce/TabularSemanticParsing>.

1 Introduction

Text-to-SQL semantic parsing addresses the problem of mapping natural language utterances to executable relational DB queries. Early work in this area focus on training and testing the semantic parser on a single DB (Hemphill et al., 1990; Dahl et al., 1994; Zelle and Mooney, 1996; Zettlemoyer and Collins, 2005; Dong and Lapata, 2016). However, DBs are widely used in many domains and

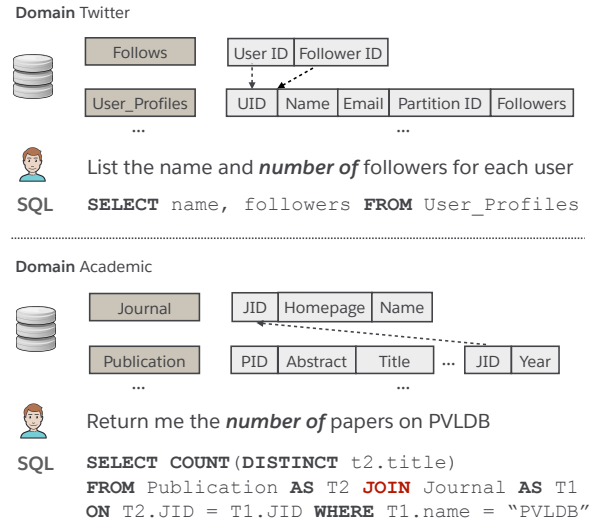


Figure 1: Two questions from the Spider dataset with similar intent resulted in completely different SQL logical forms on two DBs. In cross-DB text-to-SQL semantic parsing, the interpretation of a natural language question is strictly grounded in the underlying relational DB schema.

developing a semantic parser for each individual DB is unlikely to scale in practice.

More recently, large-scale datasets consisting of hundreds of DBs and the corresponding question-SQL pairs have been released (Yu et al., 2018; Zhong et al., 2017; Yu et al., 2019b,a) to encourage the development of semantic parsers that can work well across different DBs (Guo et al., 2019; Bogin et al., 2019b; Zhang et al., 2019; Wang et al., 2019; Suhr et al., 2020; Choi et al., 2020). The setup is challenging as it requires the model to interpret a question conditioned on a relational DB unseen during training and accurately express the question intent via SQL logic. Consider the two examples shown in Figure 1, both questions have the intent to count, but the corresponding SQL queries are drastically different due to differences in the target

DB schema. As a result, cross-DB text-to-SQL semantic parsers cannot trivially memorize seen SQL patterns, but instead has to accurately model the natural language question, the target DB structure, and the contextualization of both.

State-of-the-art cross-DB text-to-SQL semantic parsers adopt the following design principles to address the aforementioned challenges. First, the question and schema representation are contextualized with each other (Hwang et al., 2019; Guo et al., 2019; Wang et al., 2019; Yin et al., 2020). Second, pre-trained language models (LMs) such as BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019c) can significantly boost parsing accuracy by enhancing generalization over natural language variations and capturing long-term dependencies (Shaw et al., 2020). Third, as much as data privacy allows, leveraging available DB content improves understanding of the DB schema (Bogin et al., 2019b; Wang et al., 2019; Yin et al., 2020). Consider the second example in Figure 1, knowing “PLVDB” is a value of the field `Journal.Name` helps the model to generate the `WHERE` condition.

We introduce BRIDGE, a powerful sequential text-DB encoding framework assembling the three design principles mentioned above. BRIDGE represents the relational DB schema as a tagged sequence concatenated to the question. In contrast to previous work which proposed task-specific layers for modeling the DB schema (Bogin et al., 2019a,b; Zhang et al., 2019; Choi et al., 2020) and joint text-DB linking (Guo et al., 2019; Wang et al., 2019), BRIDGE encodes the tagged sequence with BERT and lightweight subsequent layers – two single-layer bi-directional LSTMs (Hochreiter and Schmidhuber, 1997). Each schema component (table or field) is simply represented using the hidden state corresponding to its special token in the hybrid sequence. To better align the schema components with the question, BRIDGE augments the hybrid sequence with *anchor texts*, which are automatically extracted DB cell values mentioned in the question. Anchor texts are appended to their corresponding fields in the hybrid sequence (Figure 2). The text-DB alignment is then implicitly achieved via fine-tuned BERT attention between overlapped lexical tokens.

Combined with a pointer-generator decoder (See et al., 2017) and *schema-consistency driven search space pruning*, BRIDGE achieves performances comparable to or better than the state-of-the-art

on the Spider (71.1% dev, 67.5% test with ensemble model) and WikiSQL (92.6% dev, 91.9% test) benchmarks, outperforming most of lately proposed models with task-specific architectures.¹ Through in-depth model comparison and error analysis, we show the proposed architecture is effective for generalizing over natural language variations and memorizing structural patterns, but struggles in compositional generalization and suffers from lack of explainability. This leads us to conclude that cross-domain text-to-SQL still poses many unsolved challenges, requiring models to demonstrate generalization over both natural language variation and structure composition while training data is often sparse.

2 Model

In this section, we present the BRIDGE model that combines a BERT-based encoder with a sequential pointer-generator to perform end-to-end cross-DB text-to-SQL semantic parsing.

2.1 Problem Definition

We formally defined the cross-DB text-to-SQL task as the following. Given a natural language question Q and the schema $\mathcal{S} = \langle \mathcal{T}, \mathcal{C} \rangle$ for a relational database, the parser needs to generate the corresponding SQL query Y . The schema consists of tables $\mathcal{T} = \{t_1, \dots, t_N\}$ and fields $\mathcal{C} = \{c_{11}, \dots, c_{1|T_1|}, \dots, c_{n1}, \dots, c_{N|T_N|}\}$. Each table t_i and each field c_{ij} has a textual name. Some fields are *primary keys*, used for uniquely indexing each data record, and some are *foreign keys*, used to reference a primary key in a different table. In addition, each field has a *data type*, $\tau \in \{\text{number, text, time, boolean, etc.}\}$.

Most existing solutions for this task do not consider DB content (Zhong et al., 2017; Yu et al., 2018). Recent approaches show accessing DB content can significantly improve system performance (Liang et al., 2018; Wang et al., 2019; Yin et al., 2020). We consider the setting adopted by Wang et al. (2019), where the model has access to the value set of each field instead of full DB content. For example, the field `Property_Type_Code` in Figure 2 can take one of the following values: {“Apartment”, “Field”, “House”, “Shop”, “Other”}.

¹An earlier version of this model is implemented within the Photon system demonstration <https://naturalsql.com> (Zeng et al., 2020), with up to one anchor text per field and a less accurate anchor text matching algorithm.

We call such value sets *picklists*. This setting protects individual data record and sensitive fields such as user IDs or credit numbers can be hidden.

2.2 Question-Schema Serialization and Encoding

As shown in Figure 2, we represent each table with its table name followed by its fields. Each table name is preceded by the special token [T] and each field name is preceded by [C]. The representations of multiple tables are concatenated to form a serialization of the schema, which is surrounded by two [SEP] tokens and concatenated to the question. Finally, following the input format of BERT, the question is preceded by [CLS] to form the hybrid question-schema serialization

$$X = [\text{CLS}], Q, [\text{SEP}], [\text{T}], t_1, [\text{C}], c_{11} \dots, c_{1|T_1|}, \\ [\text{T}], t_2, [\text{C}], c_{21}, \dots, [\text{C}], c_{N|T_N|}, [\text{SEP}].$$

X is encoded with BERT, followed by a bi-directional LSTM to form the base encoding $\mathbf{h}_X \in \mathbb{R}^{|X| \times n}$. The question segment of $\mathbf{h}_X$ is passed through another bi-LSTM to obtain the question encoding $\mathbf{h}_Q \in \mathbb{R}^{|Q| \times n}$. Each table/field is represented using the slice of $\mathbf{h}_X$ corresponding to its special token [T]/[C].

Meta-data Features We train dense look-up features to represent meta-data of the schema. This includes whether a field is a primary key ($\mathbf{f}_{\text{pri}} \in \mathbb{R}^{2 \times n}$), whether the field appears in a foreign key pair ($\mathbf{f}_{\text{for}} \in \mathbb{R}^{2 \times n}$) and the data type of the field ($\mathbf{f}_{\text{type}} \in \mathbb{R}^{|\tau| \times n}$). These meta-data features are fused with the base encoding of the schema component via a feed-forward layer $g (\mathbb{R}^{4n} \rightarrow \mathbb{R}^n)$ to obtain the following encoding output:

$$\mathbf{h}_S^{t_i} = g([\mathbf{h}_X^p; \mathbf{0}; \mathbf{0}; \mathbf{0}]), \quad (1)$$

$$\mathbf{h}_S^{c_{ij}} = g([\mathbf{h}_X^q; \mathbf{f}_{\text{pri}}^u; \mathbf{f}_{\text{for}}^v; \mathbf{f}_{\text{type}}^w]) \quad (2)$$

$$= \text{ReLU}(\mathbf{W}_g[\mathbf{h}_X^m; \mathbf{f}_{\text{pri}}^u; \mathbf{f}_{\text{for}}^v; \mathbf{f}_{\text{type}}^w] + \mathbf{b}_g) \\ \mathbf{h}_S = [\mathbf{h}^{t_1}, \dots, \mathbf{h}^{t_{|T|}}, \mathbf{h}^{c_{11}}, \dots, \mathbf{h}^{c_{N|T_N|}}] \in \mathbb{R}^{|S| \times n}, \quad (3)$$

where p is the index of [T] associated with table t_i in X and q is the index of [C] associated with field c_{ij} in X . u, v and w are feature indices indicating the properties of c_{ij} . $[\mathbf{h}_X^m; \mathbf{f}_{\text{pri}}^u; \mathbf{f}_{\text{for}}^v; \mathbf{f}_{\text{type}}^w] \in \mathbb{R}^{4n}$ is the concatenation of the four vectors. The meta-data features are specific to fields and the table representations are fused with place-holder $\mathbf{0}$ vectors.

2.3 Bridging

Modeling only the table/field names and their relations is not always enough to capture the semantics of the schema and its dependencies with the question. Consider the example in Figure 2, *Property_Type_Code* is a general expression not explicitly mentioned in the question, and without access to the set of possible field values, it is difficult to associate “houses” and “apartments” with it. To resolve this problem, we make use of *anchor text* to link value mentions in the question with the corresponding DB fields. We perform fuzzy string match between Q and the picklist of each field in the DB. The matched field values (anchor texts) are inserted into the question-schema representation X , succeeding the corresponding field names and separated by the special token [V]. If multiple values were matched for one field, we concatenate all of them in matching order (Figure 2). If a question mention is matched with values in multiple fields. We add all matches and let the model learn to resolve ambiguity.²

The anchor texts provide additional lexical clues for BERT to identify the corresponding mention in Q . And we name this mechanism “bridging”.

2.4 Decoder

We use an LSTM-based pointer-generator (See et al., 2017) with multi-head attention (Vaswani et al., 2017) as the decoder. The decoder is initiated with the final state of the question encoder. At each step, the decoder performs one of the following actions: generating a token from the vocabulary $\mathcal{V}$, copying a token from the question Q or copying a schema component from $\mathcal{S}$.

Mathematically, at each step t , given the decoder state $\mathbf{s}_t$ and the encoder representation $[\mathbf{h}_Q; \mathbf{h}_S] \in \mathbb{R}^{(|Q|+|S|) \times n}$, we compute the multi-head attention as defined in Vaswani et al. (2017):

$$e_{ij}^{(h)} = \frac{\mathbf{s}_t \mathbf{W}_U^{(h)} (\mathbf{h}_j \mathbf{W}_V^{(h)})^\top}{\sqrt{n/H}}; \quad \alpha_{ij}^{(h)} = \text{softmax}_j \{e_{ij}^{(h)}\} \quad (4)$$

$$\mathbf{z}_t^{(h)} = \sum_{j=1}^{|Q|+|S|} \alpha_{ij}^{(h)} (\mathbf{h}_j \mathbf{W}_V^{(h)}); \quad \mathbf{z}_t = [\mathbf{z}_t^{(1)}; \dots; \mathbf{z}_t^{(H)}], \quad (5)$$

where $h \in [1, \dots, H]$ is the head number and H is the total number of heads.

²This approach may over-match anchor texts from fields other than those appeared in the correct SQL query, but keeping the additional matches in X may provide useful signal rather than noise.

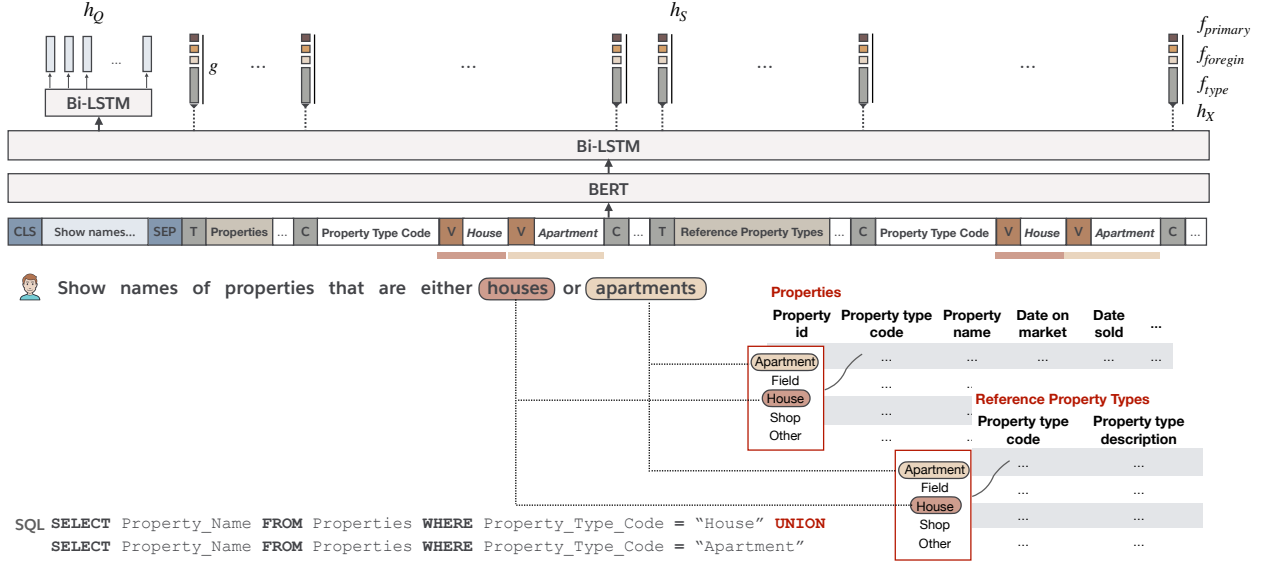


Figure 2: The BRIDGE encoder. The two phrases “houses” and “apartments” in the input question both matched to two DB fields. The matched values are appended to the corresponding field names in the hybrid sequence.

The probability of generating from $\mathcal{V}$ and the output distribution is defined as

$$p_{\text{gen}}^t = \text{sigmoid}(s_t W_{\text{gen}}^s + z_t W_{\text{gen}}^z + b_{\text{gen}}) \quad (6)$$

$$p_{\text{out}}^t = p_{\text{gen}}^t P_{\mathcal{V}}(y_t) + (1 - p_{\text{gen}}^t) \sum_{j: \tilde{X}_j = y_t} \alpha_{tj}^{(H)}, \quad (7)$$

where $P_{\mathcal{V}}(y_t)$ is the softmax LSTM output distribution and $\tilde{X}$ is the length- $(|Q| + |S|)$ sequence that consists of only the question words and special tokens [T] and [C] from X . We use the attention weights of the last head to compute the pointing distribution³.

We extend the input state to the LSTM decoder using the *selective read* proposed by Gu et al. (2016). The technical details of this extension can be found in §A.2.

2.5 Schema-Consistency Guided Decoding

We propose simple heuristics for pruning the search space of the sequence decoders, based on SQL syntax constraints and the fact that the DB fields appeared in each SQL clause must only come from the tables in the FROM clause.

Generating SQL Clauses in Execution Order

We rearrange the clauses of each SQL query in the training set into the standard DB execution order (Rob and Coronel, 1995) shown in table 1. For example, the SQL `SELECT COUNT(*)`

³In practice we find this approach better than using just one head or the average of multiple head weights (§A.4).

Written:	SELECT FROM WHERE GROUPBY HAVING ORDERBY LIMIT
Exec:	FROM WHERE GROUPBY HAVING SELECT ORDERBY LIMIT

Table 1: The written order vs. execution order of all SQL clauses appeared in Spider.

FROM Properties is converted⁴ to FROM Properties SELECT COUNT(*).

We can show that a SQL query with clauses in execution order satisfies the following lemma.

Lemma 1 *Let Y_{exec} be a SQL query with clauses arranged in execution order, then any table field in Y_{exec} must appear after the table.*

As a result, we adopt a binary attention mask ξ

$$\tilde{\alpha}_i^{(H)} = \alpha_i^{(H)} \cdot \xi \quad (8)$$

which initially has entries corresponding to all fields set to 0. Once a table t_i is decoded, we set all entries in ξ corresponding to $\{c_{i1}, \dots, c_{i|T_i|}\}$ to 1. This allows the decoder to only search in the space specified by the condition in Lemma 1 with little overhead in decoding speed.

In addition, we observe that a valid SQL query satisfies the following token transition lemma.

Lemma 2 Token Transition: *Let Y be a valid SQL query, then any table/field token in Y can only appear after a SQL reserved token; any value token in Y can only appear after a SQL reserved token or a value token.*

⁴More complex examples can be found in Table A1.

We use this heuristics to prune the set of candidate tokens at each decoding step. It is implemented via vocabulary masking.

3 Related Work

Text-to-SQL Semantic Parsing Recently the field has witnessed a re-surge of interest for text-to-SQL semantic parsing (Androutsopoulos et al., 1995), by virtue of newly released large-scale datasets (Zhong et al., 2017; Yu et al., 2018; Zhang et al., 2019) and matured neural network modeling tools (Vaswani et al., 2017; Shaw et al., 2018; Devlin et al., 2019). While existing models have surpassed human performance on benchmarks consisting of single-table and simple SQL queries (Hwang et al., 2019; Lyu et al., 2020; He et al., 2019a), ample space of improvement still remains for the Spider benchmark⁵ which consists of relational DBs and complex SQL queries.

Recent architectures proposed for this problem show increasing complexity in both the encoder and the decoder (Guo et al., 2019; Wang et al., 2019; Choi et al., 2020; Furrer et al., 2020). Bogin et al. (2019a,b) proposed to encode relational DB schema as a graph and also use the graph structure to guide decoding. Guo et al. (2019) proposes schema-linking and SemQL, an intermediate SQL representation customized for questions in the Spider dataset which was synthesized via a tree-based decoder. Wang et al. (2019) proposes RAT-SQL, a unified graph encoding mechanism which effectively covers relations in the schema graph and its linking with the question. The overall architecture of RAT-SQL is deep, consisting of 8 relational self-attention layers (Shaw et al., 2018) on top of BERT-large. In comparison, BRIDGE uses BERT combined with minimal subsequent layers. It uses a simple sequence decoder with search space-pruning heuristics and applies little abstraction to the SQL surface form.

Seq2Seq Models for Text-to-SQL Semantic Parsing Many work have applied sequence-to-sequence models to solve semantic parsing, treating it as a translation problem (Dong and Lapata, 2016; Lin et al., 2018). Text-to-SQL models take both the natural language question and the DB as input, and a commonly used input representation in existing work is to concatenate the question with a sequential version of the DB schema (or table header if there

is only a single table). Zhong et al. (2017) proposed the Seq2SQL model which first adopted this representation and tested it on WikiSQL. Hwang et al. (2019) first demonstrated that encoding such representation with BERT can achieve upperbound performance on the WikiSQL benchmark. Our work shows that such sequence representation encoded with BERT is also effective for synthesizing complex SQL queries issued to multi-table databases. Concurrently, Suhr et al. (2020) adopted a transformer model with BERT as encoder on Spider; Shaw et al. (2020) shows that the T5 model (Raffel et al., 2020) with 3 billion parameters achieves the state-of-the-art performance on Spider. However, both of these two models do not use DB content. In addition, BRIDGE achieves comparable performance with a significantly smaller model. Especially, the BRIDGE decoder is a single-layer LSTM compared to the 12-layer transformer in T5.

Text-to-SQL Semantic Parsing with DB Content Yavuz et al. (2018) uses question-value matches to achieve high-precision condition predictions on WikiSQL. Shaw et al. (2019) also shows that value information is critical to the cross-DB semantic parsing tasks, yet the paper reported negative results augmenting an GNN encoder with BERT and the overall model performance is much below state-of-the-art. While previous work such as Guo et al. (2019); Wang et al. (2019); Yin et al. (2020) use feature embeddings or relational attention layers to explicitly model schema linking, BRIDGE models the linking implicitly with BERT and lexical anchors.

In addition, instead of directly taking DB content as input, some models leverage the content by training the model with SQL query execution (Zhong et al., 2017) or performing execution-guided decoding during inference (Wang et al., 2018). To our best knowledge, such methods have been tested exclusively on the WikiSQL benchmark.

Joint Representation of Textual-Tabular Data and Pre-training BRIDGE is a general framework for jointly representing question, DB schema and the relevant DB cells. It has the potential to be applied to a wider range of problems that requires joint textual-tabular data understanding. Recently, Yin et al. (2020) proposes TaBERT, an LM for jointly representing textual and tabular data pre-trained over millions of web tables. Similarly, Herzig et al. (2020) proposes TAPas, a pre-

⁵<https://yale-lily.github.io/spider>

	Spider			WikiSQL	
	# Q	# SQL	#DB	# Q	# Table
Train	8,695	4,730	140	56,355	17,984
Dev	1,034	564	20	8,421	1,621
Test	2,147	—	40	15,878	2,787

Table 2: Text-to-SQL Dataset Statistics

trained text-table LM that supports arithmetic operations for weakly supervised table QA. Both TaBERT and TAPAS focus on contextualizing text with a single table. TaBERT was applied to Spider by encoding each table individually and modeling cross-table correlation through hierarchical attention. In comparison, BRIDGE serialized the relational DB schema and uses BERT to model cross-table dependencies. TaBERT adopts the “content snapshot” mechanism which retrieves table rows most similar to the input question and jointly encodes them with the table header. Compared to BRIDGE which uses the anchor texts, table rows are not always available if DB content access is restricted. Furthermore, anchor texts provide more focused signals that link the text and the DB schema.

4 Experiment Setup

4.1 Dataset

We evaluate BRIDGE using two well-studied cross-database text-to-SQL benchmark datasets: Spider (Yu et al., 2018) and WikiSQL (Zhong et al., 2017). Table 2 shows the statistics of the train/dev/test splits of the datasets. In the Spider benchmark, the train/dev/test databases do not overlap, and the test set is hidden from public. For WikiSQL, 49.6% of its dev tables and 45.1% of its test tables are not found in the train set. Therefore, both datasets necessitates the ability of models to generalize to unseen schema.

We run hyperparameter search and analysis on the dev set and report the test set performance only using our best approach.

4.2 Evaluation Metrics

We report the official evaluation metrics proposed by the Spider and WikiSQL authors.

Exact Match (EM) This metrics checks if the predicted SQL exactly matches the ground truth SQL. It is a performance lower bound as a semantically correct SQL query may differ from the ground truth SQL query in surface form.

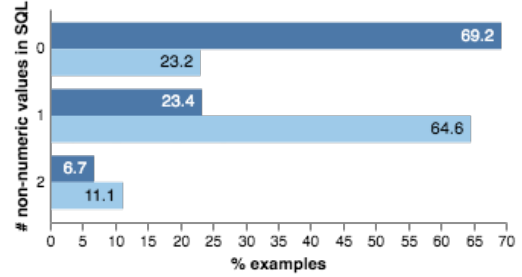


Figure 3: Distribution of # non-numeric values in the ground truth SQL queries in the Spider and WikiSQL dev sets.

Exact Set Match (E-SM) This metrics evaluates the structural correctness of the predicted SQL by checking the orderless set match of each SQL clause in the predicted query w.r.t. the ground truth. It ignores errors in the predicted values.

Execution Accuracy (EA) This metrics checks if the predicted SQL is executable on the target DB and if the execution results of match those of the ground truth. It is a performance upper bound as two SQL queries with different semantics can execute to the same results on a DB.

4.3 Implementation Details

Anchor Text Selection Given a DB, we compute the picklist of each field using the official DB files. We developed a fuzzy matching algorithm to match a question to possible value mentions in the DB (described in detail in §A.3). We include up to k matches per field, and break ties by taking the longer match. We exclude all number matches as a number mention in the question may not correspond to a DB cell (e.g. it could be a hypothetical threshold as in “shoes lower than \$50”) or cannot effectively discriminate between different fields.

Figure 3 shows the distribution of non-numeric values in the ground truth SQL queries from the Spider and WikiSQL dev sets. For Spider, 31% of the examples contain one or more non-numeric values in the ground truth queries and can potentially benefit from the bridging mechanism. For WikiSQL the ratio is significantly higher, with 76.8% of the ground truth SQL queries contain one or more non-numeric values. On both datasets, the proportion of ground truth SQL queries containing > 2 non-numeric values are negligible (0.8% for Spider and 1.1% for WikiSQL). Based on this analysis, we set $k = 2$ in all our experiments.

Data Repair The original Spider dataset contains errors in both the example files and database files. We manually corrected some errors in the train and dev examples. For comparison with other models in §5.1, we report metrics using the official dev/test sets. For our own ablation study and analysis, we report metrics using the corrected dev files. We also use a high-precision heuristics to identify missing foreign key pairs in the databases and combine them with the released ones during training and inference: if two fields of different tables have identical name and one of them is a primary key, we count them as a foreign key pair⁶.

Training We train our model using cross-entropy loss. We use Adam-SGD (Kingma and Ba, 2015) with default parameters and a mini-batch size of 32. We use the uncased BERT-large model from the Huggingface’s transformer library (Wolf et al., 2019). We set all LSTMs to 1-layer and use 8-head attention between the encoder and decoder.

- Spider: We set the LSTM hidden layer dimension to 400. We train a maximum of 100k steps. We set the learning rate to $5e^{-4}$ in the first 5,000 iterations and shrink it to 0 with the L-inv function (§A.5). We fine-tune BERT with a fine-tuning rate linearly increasing from $3e^{-5}$ to $6e^{-5}$ in the first 4,000 iterations and decaying to 0 according to the L-inv function. We randomly permute the table order in a DB schema and drop one table which does not appear in the ground truth with probability 0.3 in every training step. The training time of our model on an NVIDIA A100 GPU is approximately 51.5h (including intermediate results verification time).
- WikiSQL: We set the LSTM hidden layer dimension to 512. We train a maximum of 50k steps and set the learning rate to $5e^{-4}$ in the first 4,000 iterations and shrink it to 0 with the L-inv function. We fine-tune BERT with a fine-tuning rate linearly increasing from $3e^{-5}$ to $6e^{-5}$ in the first 4,000 iterations and decaying to 0 according to the L-inv function. The training time of our model on an NVIDIA A100 GPU is approximately 6h (including intermediate results verification time).

Decoding The decoder uses a generation vocabulary consisting of 70 SQL keywords and reserved

⁶We exclude common field names such as “name”, “id” and “code” in this procedure.

tokens, plus the 10 digits to generate numbers not explicitly mentioned in the question (e.g. “first”, “second”, “youngest” etc.). We use a beam size of 64 for leaderboard evaluation. All other experiments uses a beam size of 16. We use schema-consistency guided decoding during inference only. It cannot guarantee schema consistency⁷ and we run a static SQL correctness check on the beam search output to eliminate predictions that are either syntactically incorrect or violates schema consistency⁸. For WikiSQL, the static check also makes sure that the output query conforms to the SQL sketch used to create the dataset (Zhong et al., 2017). If no predictions in the beam satisfy the two criteria, we output a default SQL query which count the number of entries in the first table.

5 Results

5.1 End-to-end Performance Evaluation

5.1.1 Spider

Table 3 shows the performance of BRIDGE compared to other approaches ranking at the top of the Spider leaderboard. BRIDGE v1 is our model described in the original version of the paper. Comparing to BRIDGE v1, the current model is trained with BERT-large with an improved anchor text matching algorithm (§A.3). BRIDGE_L performs very competitively, significantly outperforming most of recently proposed architectures with more complicated, task-specific layers (Global-GNN, EditSQL+BERT, IRNet+BERT, RAT-SQL v2, RYANSQL+BERT_L). It also performs better than or comparable to models that explicitly model compositionality in the decoder (SmBoP, RAT-SQL v3L+BERT_L, RYANSQL).⁹ In addition, BRIDGE

⁷Consider the example SQL query shown in Table A2 which satisfies the condition of Lemma 1, the table VOTING_RECORD only appears in the first sub-query, and the field VOTING_RECORD.PRESIDENT_Vote in the second sub-query is out of scope.

⁸Prior work such as (Wang et al., 2018) performs the more aggressive execution-guided decoding. However, it is difficult to apply this approach to complex SQL queries (Zhong et al., 2017). We build a static SQL analyzer on top of the Mozilla SQL Parser (<https://github.com/mozilla/moz-sql-parser>). Our static checking approach handles complex SQL queries and avoids DB execution overhead.

⁹Simply comparing the leaderboard performances does not allow precise gauging of different modeling trade-offs, as all leaderboard entries adopt some customized pre- and post-processing of the data. For example, the schema-consistency guided decoding adopted by BRIDGE is complementary to other models. BRIDGE synthesizes a complete SQL query while several other models do not synthesize values and synthesize the FROM clause in a post-processing step (Wang et al., 2019).

Model	Dev	Test
Global-GNN (Bogin et al., 2019b) ♠	52.7	47.4
EditSQL + BERT (Zhang et al., 2019)	57.6	53.4
GNN + Bertrand-DR (Kelkar et al., 2020)	57.9	54.6
IRNet + BERT (Guo et al., 2019)	61.9	54.7
RAT-SQL v2 ♠ (Wang et al., 2019)	62.7	57.2
RYANSQL + BERT _L (Choi et al., 2020)	66.6	58.2
SmBoP + BART (Rubin and Berant, 2020)	66.0	60.5
RYANSQL v2 + BERT _L ◊	70.6	60.6
RAT-SQL v3 + BERT _L ♠ (Wang et al., 2019)	69.7	65.6
BRIDGE v1 ♠ ♥ (Lin et al., 2020)	65.5	59.2
BRIDGE _L (ours) ♠ ♥	70.0	65.0
BRIDGE _L (ours, ensemble) ♠ ♥	71.1	67.5

Model	Dev	Test
AuxNet + BART ◊ ♠ ♥	-	62.6
BRIDGE v1 ♠ ♥ (Lin et al., 2020)	65.3	59.9
BRIDGE _L (ours) ♠ ♥	68.0	64.3
BRIDGE _L (ours, ensemble) ♠ ♥	70.3	68.3

Table 3: Exact set match (top) and execution accuracy (bottom) on the Spider dev and test sets, compared to the other top-performing approaches on the leaderboard as of Dec 20, 2020. The test set results were issued by the Spider team. BERT_L denotes BERT_{LARGE}. ◊ denotes approaches without reference in publication. ♠ denotes approaches using DB content. ♥ denote approaches generating executable output.

generates executable SQL queries by copying values from the input question while most existing models only predicts the SQL syntax skeleton.¹⁰ As of Dec 20, 2020, BRIDGE ranks top-1 on the Spider leaderboard by execution accuracy.

RAT-SQL v3+BERT_L outperforms BRIDGE in terms of exact set match with a small margin. We further look at the performance comparison between the two models across different SQL query hardness level (Table 4). Overall, BRIDGE outperforms RAT-SQL v3+BERT_L in the easy category but underperforms it in the other three categories, with considerable gaps in medium and hard.

We hypothesize that differences in both the encoders and decoders of the two models have contributed to the performance differences. The RAT-SQL encoder and decoder are designed with compositional inductive bias. It models the relational DB schema as a graph encoded with relational self-attention. The decoder uses SQL-syntax guided generation (Yin and Neubig, 2017). BRIDGE, on the other hand, adopts a Seq2Seq architecture. In addition, RAT-SQL v3 models the lexical mapping

¹⁰We believe the execution accuracy can be further improved by having the model copying the anchor texts and plan to explore this in future work.

Model	Easy	Medium	Hard	Ex-Hard	All
count	250	440	174	170	1034
<i>Dev</i>					
RAT-SQL v3+B _L ♠	86.4	73.6	62.1	42.9	69.7
BRIDGE _L ♠	89.1	72.2	56.3	50.0	70.0
BRIDGE _{L,ens} ♠	89.1	71.7	62.1	51.8	71.1
<i>Test</i>					
IRNet+B	77.2	58.7	48.1	25.3	54.7
BRIDGE _L ♠	85.1	71.2	55.3	36.1	65.0
RAT-SQL v3+B _L ♠	83.0	71.3	58.3	38.4	65.6
BRIDGE _{L,ens} ♠	85.3	73.4	59.6	40.3	67.5

Table 4: E-SM by SQL hardness level compared to other approaches on Spider leaderboard.

between question-schema and question-value via a graph with edge labeled by the matching condition (full-word match, partial match, etc.). BRIDGE represents the same information in a tagged sequence and uses fine-tuned BERT to implicitly obtain such mapping. While the anchor text selection algorithm (§4.3) has taken into account string variations, BERT may not be able to capture the linking when string variations exist – it has not seen tabular input during pre-training. The tokenization scheme adopted by BERT and other pre-trained LMs (e.g. GPT-2) cannot effectively capture partial string matches in a novel input (e.g. “cats” and “cat” are two different words in the vocabularies of BERT and GPT-2). Pre-training the architecture using more tables and heuristically aligned text may alleviate this problem (Yin et al., 2020; Herzig et al., 2020). Finally, we notice that ensembling three models (averaging the output distributions at each decoding step) trained with different random seeds improves the performance in all SQL hardness levels, especially in *medium*, *hard* and *extra-hard*.

5.1.2 WikiSQL

Table 5 reports the comparison of BRIDGE_L to other top-performing entries on the WikiSQL leaderboard. BRIDGE_L achieves SOTA performance on WikiSQL, surpassing the widely cited SQLova model (Hwang et al., 2019) by a significant margin. Among the baselines shown in Table 5, SQLova is the one that’s strictly comparable to BRIDGE as both use BERT-large-uncased.¹¹ Leveraging table content (anchor texts) enables

¹¹NL2SQL uses BERT-based-uncased. Hydra-Net uses RoBERTa-Large (Liu et al., 2019a) and X-SQL uses MT-DNN (Liu et al., 2019b).

Model	Dev		Test	
	EM	EX	EM	EX
SQLova (Hwang et al., 2019)	81.6	87.2	80.7	86.2
X-SQL (He et al., 2019b)	83.8	89.5	83.3	88.7
IE-SQL (Ma et al., 2020)	84.6	88.7	84.6	88.8
NL2SQL $\spadesuit$ (Guo and Gao, 2019)	84.3	90.3	83.7	89.2
HydraNet (Lyu et al., 2020)	83.6	89.1	83.8	89.2
BRIDGE _L $\spadesuit$	86.2	91.7	85.7	91.1
SQLova+EG (Hwang et al., 2019)	84.2	90.2	83.6	89.6
NL2SQL+EG $\spadesuit$ (Guo and Gao, 2019)	85.4	91.1	84.5	90.1
X-SQL+EG (He et al., 2019b)	86.2	92.3	86.0	91.8
BRIDGE _L +EG $\spadesuit$	86.8	92.6	86.3	91.9
HydraNet+EG (Lyu et al., 2020)	86.6	92.4	86.5	92.2
IE-SQL+EG (Ma et al., 2020)	87.9	92.6	87.8	92.5

Table 5: Comparison between BRIDGE and other top-performing models on the WikiSQL leaderboard as of Dec 20, 2020. $\spadesuit$ denotes approaches using DB content. +EG denotes approaches using execution-guided decoding (Wang et al., 2018).

BRIDGE_L to be the best-performing model without execution-guided (EG) decoding (Wang et al., 2018). However, comparing to SQLova, X-SQL and HydraNet, BRIDGE benefits noticeably less from EG. A probably reason for this is that the schema-consistency guided decoding already ruled out a significant number of SQL queries that will raise errors during execution. In addition, all models leveraging DB content during training (BRIDGE and NL2SQL) benefit less from EG.

5.2 Ablation Study

Spider We perform a thorough ablation study to show the contribution of each BRIDGE sub-component (Table 6). The decoding search space pruning strategies we introduced (including schema-consistency guided decoding and static SQL correctness check) are effective, with absolute E-SM improvements 0.3% on average. On the other hand, encoding techniques for jointly representing textual and tabular input contribute more. Especially, the bridging mechanism results in an absolute E-SM improvement of 1.6%. A further comparison between BRIDGE with and without bridging at different SQL hardness levels (Table 6) shows that the technique is especially effective at improving the model performance in the extra-hard category. We also did a fine-grained ablation study on the bridging mechanism, by inserting only the special token [v] into the hybrid sequence without the anchor texts. The average model performance is not hurt and the variance decreased. This indicates that the [v] tokens act as markers for columns

Model	Exact Set Match (%)	
	Mean	Max
BRIDGE _L	68.2 $\pm$ 1.0	69.1
- SC-guided decoding	67.9 $\pm$ 0.7 (-0.3)	69.1 (-0.0)
- static SQL check	67.9 $\pm$ 0.6 (-0.3)	68.8 (-0.3)
- anchor text	68.3 $\pm$ 0.4 (+0.1)	68.8 (-0.3)
- table shuffle & drop	67.5 $\pm$ 1.0 (-0.7)	68.7 (-0.4)
- meta data	67.2 $\pm$ 0.2 (-1.0)	67.4 (-1.7)
- bridging	66.6 $\pm$ 0.5 (-1.6)	67.3 (-1.8)
- BERT	17.7 $\pm$ 0.7 (-50.5)	18.3(-50.8)

Model	Easy	Medium	Hard	Ex-Hard	All
count	250	440	174	170	1034
BRIDGE _L	85.5	71.5	56.3	51.8	69.1
-bridging	86.3	70.0	56.9	42.8	67.3

Table 6: BRIDGE ablations on the Spider dev set. We report the E-SM of each model variations averaged over 3 runs in the main study (top); and the E-SM of the best model in each variation in the study by SQL hardness (bottom).

whose value matched with the input question and contribute to a significant proportion of the performance improvement by bridging.¹² However, since the full model attained the best performance on the dev set, we keep the anchor texts in our representation.

We also observe that the dense meta data feature encoding (§ 2.2) is helpful, resulting in 1% absolute improvement on average. Shuffling and randomly dropping non-ground-truth tables during training also mildly helps our approach, as it increases the variation of DB schema seen by the model and reduces overfitting to a particular table arrangement. Furthermore, BERT is critical to the performance of BRIDGE, magnifying performance of the base model by more than three folds. This is considerably larger than the improvement prior approaches have obtained from adding BERT. Consider the performances of RAT-SQL v2 and RAT-SQL v2+BERT_L in Table 3, the improvement using BERT_L is 7%. This shows that simply adding BERT to existing approaches results in significant redundancy in the model architecture. We perform a qualitative attention analysis in §A.8 to show that after fine-tuning, the BERT layers effectively capture the linking between question mentions and the anchor texts, as well as the relational DB structures.

¹²A similar mechanism is proposed by (Yin et al., 2020), where learnable dense features are concatenated to the representations of matched utterance tokens and table/fields.

Model	w/o EG		w/ EG	
	EM	EX	EM	EX
BRIDGE _L	86.2	91.7	86.8	92.6
-bridging	82.6	88.5	84.5	90.8

Table 7: BRIDGE ablations on the WikiSQL dev set.

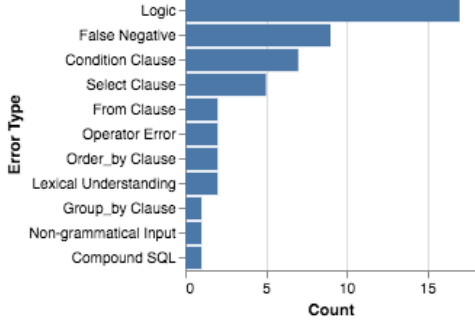


Figure 4: BRIDGE error type distribution (Spider dev).

WikiSQL The model variance on WikiSQL is much smaller than that on Spider, hence we report the ablation study results using the best model in each category. As shown in Table 7, the bridging mechanism significantly enhances the model performance, especially when execution-guided decoding is not applied. As shown in Figure 3, 76.8% of the ground truth SQL queries in the WikiSQL dev set contain at least one non-numeric values. The dataset contains simple queries and the main challenge comes from interpreting filtering conditions in the WHERE clause (Yavuz et al., 2018). And bridging is very effective for solving this challenge.

5.3 Error Analysis

We randomly sampled 50 Spider dev set examples for which the best BRIDGE model failed to produce a prediction that matches the ground truth and manually categorized the errors. Each example is assigned to only the category it fits most.

5.3.1 Manual Evaluation

Figure 4 shows the number of examples in each category. 18% of the examined predictions are false negatives. Among them, 5 are semantically equivalent to the ground truths; 3 use GROUP BY keys different but equivalent to those of the ground truth (e.g. GROUP BY car_models.name vs. GROUP BY car_models.id); 1 has the wrong ground truth annotation. Among the true negatives, The dominant type of errors is logical mistake (18), where the output SQL query failed to represent the core logic expressed in the question. 17 have errors that can

be pinpointed to specific clauses: WHERE (7), SELECT (5), FROM (2), ORDER BY (2), GROUP BY (1). 2 have errors in the operators: 1 in the aggregation operator and 1 in the DISTINCT operator. 1 have errors in compounding SQL clauses. 2 were due to lack of lexical and commonsense knowledge when interpreting the question, e.g. *predominantly spoken language, all address lines*. 1 example has non-grammatical natural language question.

5.3.2 Qualitative Analysis

Table 8 shows examples of errors from each major error type mentioned previously.

Logic Errors Logic error is a diverse category. Frequently in this case we see the model memorizing patterns seen on the training set but failed on compositional generalization. Consider the first example in this category. Superlative relation such as “highest” is often represented in the training set by sorting the retrieved records in descending order and taking the top 1. The model memorizes this pattern and output the correct logic for *finding the stadium with the highest capacity*. It also output the correct pattern for *counting the number of concerts*. Yet the correct way of combining these two logical fragments to realize the meaning in the question is to use a nested SQL query in the WHERE condition. BRIDGE joined them flatly, and the resulting query has completely different semantics. The second example illustrates an even more interesting case. The target database is a second normal form¹³ that triggers self-join relations (the *friend* of a *highschooler* is another *highschooler*). Self-joins do not appear frequently in the dataset and we hypothesize it is very challenging for a Seq2Seq based model like BRIDGE to grasp such relation. Introducing compositional inductive bias in both the encoder and decoder could be a promising direction for solving these extra-hard cases.

Lexical Understanding Another category of errors occur when the input utterance contains unseen words or phrasal expressions. While BRIDGE builds on top of pre-trained language models such as BERT, it is especially challenging for the model to interpret these text units grounded to the DB schema. Consider the first example in this category, “predominantly” means *spoken by the largest percentage of the population*. It is almost impossible for the model to see such diverse natural language

¹³https://en.wikipedia.org/wiki/Second_normal_form

Logic	Find the number of concerts happened in the stadium with <u>the highest capacity</u> . concert_singer
	✗ SELECT COUNT(*) FROM stadium JOIN concert ON stadium.Stadium_ID = concert.Stadium_ID ORDER BY stadium.Capacity DESC LIMIT 1 ✓ SELECT COUNT(*) FROM concert WHERE stadium_id = (SELECT stadium_id FROM stadium ORDER BY capacity DESC LIMIT 1)
Lexical Understanding	Show the names of all of the high schooler Kyle's friends. network_1
	✗ SELECT Highschooler.name FROM Friend JOIN Highschooler ON Friend.friend_id = Highschooler.ID WHERE Highschooler.name = "Kyle" ✓ SELECT T3.name FROM Friend AS T1 JOIN Highschooler AS T2 ON T1.student_id = T2.id JOIN Highschooler AS T3 ON T1.friend_id = T3.id WHERE T2.name = "Kyle"
Commonsense	Count the number of countries for which Spanish is the <u>predominantly spoken language</u> . world_1
	✗ SELECT COUNT(*) FROM countrylanguage WHERE countrylanguage.Language = "Spanish" ✓ SELECT COUNT(*), MAX(Percentage) FROM countrylanguage WHERE LANGUAGE = "Spanish" GROUP BY CountryCode
Robustness	What are the full names of all left handed players, in order of birth date? WTA_1
	✗ SELECT first_name, last_name FROM players ORDER BY birth_date ✓ SELECT first_name, last_name FROM players WHERE hand = 'L' ORDER BY birth_date
Robustness	Which address holds the most number of students currently? List the address id and <u>all lines</u> . student_transcripts_tracking
	✗ SELECT Addresses.line_1, Students.current_address_id FROM Addresses JOIN Students ON Addresses.address_id = Students.current_address_id GROUP BY Students.current_address_id ORDER BY COUNT(*) DESC LIMIT 1 ✓ SELECT Addresses.address_id , Addresses.line_1 , Addresses.line_2 FROM Addresses JOIN Students ON Addresses.address_id = Students.current_address_id GROUP BY Addresses.address_id ORDER BY count(*) DESC LIMIT 1
Robustness	What is the model of the car with the smallest amount of <u>horsepower</u> ? car_1
	✗ SELECT cars_data.Horsepower FROM cars_data ORDER BY cars_data.Horsepower LIMIT 1 ✓ SELECT T1.Model FROM CAR_NAMES AS T1 JOIN CARS_DATA AS T2 ON T1.MakeId = T2.Id ORDER BY T2.horsepower ASC LIMIT 1
Robustness	What is the <u>total population and average area of countries in the continent of North America</u> whose area is bigger than 3000? concert_singer
	✗ SELECT SUM(country.Population), AVG(country.Population) FROM country WHERE country.Continent = "North America" AND country.SurfaceArea > 3000> ✓ SELECT SUM(country.population), AVG(country.surfacearea) FROM country WHERE country.Continent = "north america" and country.SurfaceArea > 3000>

Table 8: Errors cases of BRIDGE on the Spider dev set. The samples were randomly selected from the medium hardness level. ✗denotes the wrong predictions made by BRIDGE and ✓denotes the ground truths.

during supervised learning. Infusing such knowledge via pre-training is also non-trivial, but worth investigating. Continuous learning is a promising direction for this type of challenges, where the model is trained to ask clarification questions and learns new knowledge from user interaction (Yao et al., 2020).

Commonsense As shown by the example, US address contains two lines is a commonsense knowledge, but the model has difficulty inferring that “all lines” maps to “line_1 and line_2”. Again, we think continuous learning could be an effective solution for this case.

Robustness The final major category of error has to do with the model blatantly ignoring information in the utterance, even when the underlying logic is not complicated, indicating that spurious correlation was captured during training (Tu et al., 2020). Consider the first example, the model places the Horsepower field in the SELECT clause, while the question asks for “the model of the car”. In the second example, the model predicts SELECT SUM(Population), AVG(Population) while the question asks for *total population and average area of countries*. We think better modeling of compositionality in the natural language may reduce this type of errors. For example, modeling its

span structure (Joshi et al., 2019; Herzig and Berant, 2020) and constructing interpretable grounding with the DB schema.

6 Conclusion

We present BRIDGE, a powerful sequential architecture for modeling dependencies between natural language question and relational DBs in cross-DB semantic parsing. BRIDGE serializes the question and DB schema into a tagged sequence and maximally utilizes pre-trained LMs such as BERT to capture the linking between text mentions and the DB schema components. It uses anchor texts to further improve the alignment between the two cross-modal inputs. Combined with a simple sequential pointer-generator decoder with schema-consistency driven search space pruning, BRIDGE attained state-of-the-art performance on the widely used Spider and WikiSQL text-to-SQL benchmarks.

Our analysis shows that BRIDGE is effective at generalizing over natural language variations and memorizing structural patterns. It achieves the upperbound score on WikiSQL and significantly outperforms previous work in the easy category of Spider. However, it struggles in compositional generalization and sometimes makes unexplainable mistakes. This indicates that when data is ample and the target logic form is shallow, sequence-to-sequence models are good choices for cross-DB semantic parsing, especially given the implementation is easier and decoding is efficient. For solving the general text-to-SQL problem and moving towards production, we plan to further improve compositional generalization and interpretability of the model. We also plan to study the application of BRIDGE and its extensions to other tasks that requires joint textual and tabular understanding such as weakly supervised semantic parsing and fact checking.

Acknowledgements

We thank Yingbo Zhou for helpful discussions. We thank the anonymous reviewers and members of Salesforce Research for their thoughtful feedback. A significant part of the experiments were completed during the California Bay Area shelter-in-place order for COVID-19. Our heartfelt thanks go to all who worked hard to keep others safe and enjoy a well-functioning life during this challenging time. 

References

- I. Androustopoulos, G.D. Ritchie, and P. Thanisch. 1995. [Natural language interfaces to databases – an introduction](#). *Natural Language Engineering*, 1(1):29–81.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. [Longformer: The long-document transformer](#). *CoRR*, abs/2004.05150.
- Ben Bogin, Jonathan Berant, and Matt Gardner. 2019a. [Representing schema structure with graph neural networks for text-to-sql parsing](#). In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 4560–4565. Association for Computational Linguistics.
- Ben Bogin, Matt Gardner, and Jonathan Berant. 2019b. [Global reasoning over database structures for text-to-sql parsing](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pages 3657–3662. Association for Computational Linguistics.
- DongHyun Choi, Myeong Cheol Shin, EungGyun Kim, and Dong Ryeol Shin. 2020. [RYANSQL: recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases](#). *CoRR*, abs/2004.03125.
- Deborah A. Dahl, Madeleine Bates, Michael Brown, William M. Fisher, Kate Hunicke-Smith, David S. Pallett, Christine Pao, Alexander I. Rudnicky, and Elizabeth Shriberg. 1994. [Expanding the scope of the ATIS task: The ATIS-3 corpus](#). In *Human Language Technology, Proceedings of a Workshop held at Plainsboro, New Jersey, USA, March 8-11, 1994*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1*, pages 4171–4186.
- Li Dong and Mirella Lapata. 2016. [Language to logical form with neural attention](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics.
- Daniel Furrer, Marc van Zee, Nathan Scales, and Nathanael Schärli. 2020. [Compositional generalization in semantic parsing: Pre-training vs. specialized architectures](#). *CoRR*, abs/2007.08970.

- Jiatao Gu, Zhengdong Lu, Hang Li, and Victor O. K. Li. 2016. [Incorporating copying mechanism in sequence-to-sequence learning](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics.
- Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards complex text-to-sql in cross-domain database with intermediate representation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 4524–4535.
- Tong Guo and Huilin Gao. 2019. [Content enhanced bert-based text-to-sql generation](#). *CoRR*, abs/1910.07179.
- Pengcheng He, Yi Mao, Kaushik Chakrabarti, and Weizhu Chen. 2019a. [X-SQL: reinforce schema representation with context](#). *CoRR*, abs/1908.08113.
- Pengcheng He, Yi Mao, Kaushik Chakrabarti, and Weizhu Chen. 2019b. [X-sql: reinforce schema representation with context](#). *arXiv preprint arXiv:1908.08113*.
- Charles T. Hemphill, John J. Godfrey, and George R. Doddington. 1990. The ATIS spoken language systems pilot corpus. In *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, USA, June 24-27, 1990*.
- Jonathan Herzig and Jonathan Berant. 2020. [Span-based semantic parsing for compositional generalization](#). *CoRR*, abs/2009.06040.
- Jonathan Herzig, Paweł Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. 2020. [Tapas: Weakly supervised table parsing via pre-training](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Seattle, Washington, United States. To appear.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. [Long short-term memory](#). *Neural Computation*, 9(8):1735–1780.
- Wonseok Hwang, Jinyeung Yim, Seunghyun Park, and Minjoon Seo. 2019. [A comprehensive exploration on wikisql with table-aware word contextualization](#). *CoRR*, abs/1902.01069.
- Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer, and Omer Levy. 2019. [Spanbert: Improving pre-training by representing and predicting spans](#). *CoRR*, abs/1907.10529.
- Amol Kelkar, Rohan Relan, Vaishali Bhardwaj, Saurabh Vaichal, and Peter Relan. 2020. [Bertrandr: Improving text-to-sql using a discriminative re-ranker](#). *arXiv preprint arXiv:2002.00557*.
- Diederik P. Kingma and Jimmy Ba. 2015. [Adam: A method for stochastic optimization](#). In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*.
- Anna Korhonen, David R. Traum, and Lluís Màrquez, editors. 2019. *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*. Association for Computational Linguistics.
- Chen Liang, Mohammad Norouzi, Jonathan Berant, Quoc V. Le, and Ni Lao. 2018. [Memory augmented policy optimization for program synthesis and semantic parsing](#). In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3-8 December 2018, Montréal, Canada*, pages 10015–10027.
- Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. [Bridging textual and tabular data for cross-domain text-to-sql semantic parsing](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings, EMNLP 2020, Online Event, 16-20 November 2020*, pages 4870–4888. Association for Computational Linguistics.
- Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. [Nl2bash: A corpus and semantic parser for natural language interface to the linux operating system](#). In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation, LREC 2018, Miyazaki, Japan, May 7-12, 2018*. European Language Resources Association (ELRA).
- Tianyu Liu, Fuli Luo, Pengcheng Yang, Wei Wu, Baobao Chang, and Zhifang Sui. 2019a. [Towards comprehensive description generation from factual attribute-value tables](#). In (Korhonen et al., 2019), pages 5985–5996.
- Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. 2019b. [Multi-task deep neural networks for natural language understanding](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4487–4496, Florence, Italy. Association for Computational Linguistics.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019c. [Roberta: A robustly optimized BERT pretraining approach](#). *CoRR*, abs/1907.11692.
- Qin Lyu, Kaushik Chakrabarti, Shobhit Hathi, Souvik Kundu, Jianwen Zhang, and Zheng Chen. 2020. [Hybrid ranking network for text-to-sql](#). Technical Report MSR-TR-2020-7, Microsoft Dynamics 365 AI.

- Jianqiang Ma, Zeyu Yan, Shuai Pang, Yang Zhang, and Jianping Shen. 2020. [Mention extraction and linking for SQL query generation](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6936–6942, Online. Association for Computational Linguistics.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *J. Mach. Learn. Res.*, 21:140:1–140:67.
- Peter Rob and Carlos Coronel. 1995. *Database systems - design, implementation, and management (2. ed.)*. Boyd and Fraser.
- Ohad Rubin and Jonathan Berant. 2020. [Smbop: Semi-autoregressive bottom-up semantic parsing](#). *CoRR*, abs/2010.12412.
- Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 1073–1083.
- Peter Shaw, Ming-Wei Chang, Panupong Pasupat, and Kristina Toutanova. 2020. [Compositional generalization and natural language variation: Can a semantic parsing approach handle both?](#) *CoRR*, abs/2010.12725.
- Peter Shaw, Philip Massey, Angelica Chen, Francesco Piccinno, and Yasemin Altun. 2019. [Generating logical forms from graph representations of text and entities](#). In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 95–106. Association for Computational Linguistics.
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. [Self-attention with relative position representations](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)*, pages 464–468. Association for Computational Linguistics.
- Emma Strubell, Patrick Verga, Daniel Andor, David Weiss, and Andrew McCallum. 2018. [Linguistically-informed self-attention for semantic role labeling](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 5027–5038. Association for Computational Linguistics.
- Alane Suhr, Ming-Wei Chang, Peter Shaw, and Kenton Lee. 2020. Exploring unexplored generalization challenges for cross-database semantic parsing. In *The 58th annual meeting of the Association for Computational Linguistics (ACL)*.
- Lifu Tu, Garima Lalwani, Spandana Gella, and He He. 2020. [An empirical study on robustness to spurious correlations using pre-trained language models](#). *CoRR*, abs/2007.06778.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*, pages 5998–6008.
- Jesse Vig. 2019. [A multiscale visualization of attention in the transformer model](#). *arXiv preprint arXiv:1906.05714*.
- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Margot Richardson. 2019. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *ArXiv*, abs/1911.04942.
- Chenglong Wang, Po-Sen Huang, Alex Polozov, Marc Brockschmidt, and Rishabh Singh. 2018. [Execution-guided neural program decoding](#). *CoRR*, abs/1807.03100.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, R’emi Louf, Morgan Funtowicz, and Jamie Brew. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *ArXiv*, abs/1910.03771.
- Ziyu Yao, Yiqi Tang, Wen-tau Yih, Huan Sun, and Yu Su. 2020. [An imitation game for learning semantic parsers from user interaction](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 6883–6902. Association for Computational Linguistics.
- Semih Yavuz, Izzeddin Gur, Yu Su, and Xifeng Yan. 2018. [What it takes to achieve 100 percent condition accuracy on wikisql](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 1702–1711. Association for Computational Linguistics.
- Pengcheng Yin and Graham Neubig. 2017. A syntactic neural model for general-purpose code generation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 440–450.
- Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. [Tabert: Pretraining for joint understanding of textual and tabular data](#). *CoRR*, abs/2005.08314.

Tao Yu, Michihiro Yasunaga, Kai Yang, Rui Zhang, Dongxu Wang, Zifan Li, and Dragomir R. Radev. 2018. Syntaxsqlnet: Syntax tree networks for complex and cross-domain text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 1653–1663.

Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Richard Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter S. Lasecki, and Dragomir R. Radev. 2019a. *Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases*. *CoRR*, abs/1909.05378.

Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir R. Radev. 2019b. *Sparc: Cross-domain semantic parsing in context*. In (Korhonen et al., 2019), pages 4511–4523.

John M. Zelle and Raymond J. Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence and Eighth Innovative Applications of Artificial Intelligence Conference, AAAI 96, IAAI 96, Portland, Oregon, USA, August 4-8, 1996, Volume 2.*, pages 1050–1055.

Jichuan Zeng, Xi Victoria Lin, Steven C. H. Hoi, Richard Socher, Caiming Xiong, Michael R. Lyu, and Irwin King. 2020. *Photon: A robust cross-domain text-to-sql system*. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, ACL 2020, Online, July 5-10, 2020*, pages 204–214. Association for Computational Linguistics.

Luke S. Zettlemoyer and Michael Collins. 2005. *Learning to map sentences to logical form: Structured classification with probabilistic categorical grammars*. In *UAI '05, Proceedings of the 21st Conference in Uncertainty in Artificial Intelligence, Edinburgh, Scotland, July 26-29, 2005*, pages 658–666. AUAI Press.

Rui Zhang, Tao Yu, Heyang Er, Sungrok Shim, Eric Xue, Xi Victoria Lin, Tianze Shi, Caiming Xiong, Richard Socher, and Dragomir R. Radev. 2019. *Editing-based SQL query generation for cross-domain context-dependent questions*. *CoRR*, abs/1909.00786.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103.

A Appendix

A.1 Examples of SQL queries with clauses arranged in execution order

We show more examples of complex SQL queries with their clauses arranged in written order vs. execution order in Table A1.

A.2 Selective read decoder extension

The selective read operation was introduced by Gu et al. (2016). It extends the input state to the decoder LSTM with the corresponding encoder hidden states of the tokens being copied. This way the decoder was provided information on which part of the input has been copied.

Specically, we modified the input state¹⁴ of our decoder LSTM to the following:

$$y_t = [e_{t-1}; (1 - p_{\text{gen}}^t) \cdot \zeta_{t-1}] \in \mathbb{R}^{2n}, \quad (9)$$

where p_{gen}^t is the scalar probability that a token is copied at step t . $e_{t-1} \in \mathbb{R}^n$ is either the embedding of a generated vocabulary token or a learned vector indicating if a table, field or question token is copied in step $t - 1$. $\zeta_{t-1} \in \mathbb{R}^n$ is the *selective read* vector, which is a weighted sum of the encoder hidden states corresponding to the tokens copied in step $t - 1$:

$$\zeta(y_{t-1}) = \sum_{j=1}^{|Q|+|S|} \rho_{t-1,j} \mathbf{h}_j; \quad \rho_{t-1,j} = \begin{cases} \frac{1}{K} \alpha_{t-1,j}^{(H)} & \tilde{X}_j = y_{t-1} \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Here $K = \sum_{j: \tilde{X}_j = y_{t-1}} \alpha_{t-1,j}^{(H)}$ is a normalization term considering there may be multiple positions equals to y_{t-1} in $\tilde{X}$.

A.3 Anchor text selection

We convert the question and field values into lower cased character sequences and compute the longest sub-sequence match with heuristically determined matching boundaries. For example, the sentence “how many students keep cats as pets?” matches with the cell value “cat” (s_c) and the matched substring is “cat” (s_m). We further search the question starting from the start and end character indices i, j of s_m in the question to make sure that word boundaries can be detected within $i - 2$ to $j + 2$, otherwise the match is invalidated. This excludes matches

¹⁴The original formulation by Gu et al. (2016) does not contain the $(1 - p_{\text{gen}}^t)$ term in Equation 9. We introduce this term as for some tokens there is ambiguity regarding whether the token is copied or generated from the decoder vocabulary.

Written: `SELECT rid FROM routes WHERE dst_apid IN (SELECT apid FROM airports WHERE country = 'United States') AND src_apid IN (SELECT apid FROM airports WHERE country = 'United States')`
Exec: `FROM routes WHERE dst_apid IN (FROM airports WHERE country = 'United States' SELECT apid) AND src_apid IN (FROM airports WHERE country = 'United States' SELECT apid) SELECT rid`

Written: `SELECT t3.name FROM publication_keyword AS t4 JOIN keyword AS t1 ON t4.kid = t1.kid JOIN publication AS t2 ON t2.pid = t4.pid JOIN journal AS t3 ON t2.jid = t3.jid WHERE t1.keyword = "Relational Database" GROUP BY t3.name HAVING COUNT(DISTINCT t2.title) = 60`
Exec: `FROM publication_keyword AS t4 JOIN keyword AS t1 ON t4.kid = t1.kid JOIN publication AS t2 ON t2.pid = t4.pid JOIN journal AS t3 ON t2.jid = t3.jid WHERE t1.keyword = "Relational Database" GROUP BY t3.name HAVING COUNT(DISTINCT t2.title) = 60 SELECT t3.name`

Written: `SELECT COUNT(DISTINCT state) FROM college WHERE enr < (SELECT AVG(enr) FROM college)`
Exec: `FROM college WHERE enr < (FROM college SELECT AVG(enr)) SELECT COUNT(DISTINCT state)`

Written: `SELECT DISTINCT T1.LName FROM STUDENT AS T1 JOIN VOTING_RECORD AS T2 ON T1.StuID = PRESIDENT_Vote EXCEPT SELECT DISTINCT LName FROM STUDENT WHERE Advisor = "2192"`
Exec: `FROM STUDENT AS T1 JOIN VOTING_RECORD AS T2 ON T1.StuID = PRESIDENT_Vote SELECT DISTINCT T1.LName EXCEPT FROM STUDENT WHERE Advisor = 2192 SELECT DISTINCT LName`

Table A1: Examples of complex SQL queries with clauses in the normal order and the DB execution order.

`FROM STUDENT JOIN VOTING_RECORD ON STUDENT.StuID = VOTING_RECORD.PRESIDENT_Vote SELECT DISTINCT STUDENT.LName EXCEPTFROM STUDENT WHERE STUDENT.Advisor = 2192 SELECT DISTINCT VOTING_RECORD.PRESIDENT_Vote`

Table A2: An example sequence satisfies the condition of Lemma 1 but violates schema consistency. Here the field `VOTING_RECORD.PRESIDENT_Vote` in the second sub-query is out of scope.

which are sub-strings of the question words, e.g. “cat” vs. “category”. Denoting matched whole-word phrase in the question as s_q , we define the question match score and cell match score as

$$\beta_q = |s_m|/|s_q| \quad (11)$$

$$\beta_c = |s_c|/|s_q| \quad (12)$$

We define a coarse accuracy measurement to tune the question match score threshold θ_q and the cell match threshold θ_c . Namely, given the list of matched anchor texts $\mathcal{P}$ obtained using the aforementioned procedure and the list of textual values $\mathcal{G}$ extracted from the ground truth SQL query, when compute the percentage of anchor texts appeared in $\mathcal{G}$ and the percentage of values in $\mathcal{G}$ that appeared in $\mathcal{P}$ as approximated precision (p') and recall (r'). Note that this metrics does not evaluate if the matched anchor texts are associated with the correct field.

For $k = 2$, we set $\theta_q = 0.5$ and $\theta_c = 0.8$. On the training set, the resulting $p' = 73.7$, $r' = 74.9$. 25.7% examples have at least one anchor text match with 1.89 average number of matches per example among them. On the dev set, the resulting $p' = 90.0$, $r' = 92.2$. 30.9% examples have at least one

match with 1.73 average number of matches per example among them. The training set metrics are lower as some training databases do not have DB content files.

To quantify the effect of anchor text matching accuracy to the end-to-end performance, we run a set of experiments comparing BRIDGE performance w.r.t. different anchor text matching F1s. Our preliminary results show that with the same anchor text matching recall, varying the precision does not significantly change the end-to-end model performance.

A.4 Performance by number of attention heads

While multi-head attention between encoder and decoder is typically used in transformers (Vaswani et al., 2017), our experiments show they are effective for the BRIDGE model as well. Table A3 shows the performance of BRIDGE_L w.r.t. different number of attention heads, where the attention probability computed by the last head is used as the copy probability. We saw that using more than 1 heads in general significantly improves over using only 1 head, where both the 2-head and 4-head attentions give best performance.

# attn heads	Exact Set Match (%)	
	Mean	Max
1-last	67.4± 0.3	67.7
2-last	68.1± 0.8	69.1
4-last	68.1± 0.5	68.8
8-last	67.7± 0.7	68.7
2-mean	67.8± 0.6	68.8

Table A3: End-to-end performance of BRIDGE_L w.r.t. different number of attention heads between encoder and decoder. “-last” indicates the last attention head is used as the copy probability. “-mean” indicates the mean of all attention heads is used. We report the E-SM of each model averaged over 5 runs.

A.5 The Linear-inverse-square-root (L-inv) learning rate decay function

The linear ($\gamma_0 - \alpha n$) and inverse-square-root ($\frac{\gamma_0}{\sqrt{n}}$) learning rate schedulers are commonly used for learning rate decay in neural network training¹⁵. The linear one decays slower in the beginning but slower in the end. The inverse-square-root one decays faster in the beginning but approaches 0 when $n \rightarrow \text{inf}$. We hence combine the two functions and propose a new learning rate scheduler that both decays fast in the beginning and also reaches 0 with finite n . The L-inv learning rate scheduler is defined as:

$$\frac{\gamma_0}{\sqrt{n}} - \beta n,$$

where $\beta = \frac{\gamma_0}{\sqrt{n_{\max}}}$ and $n_{\max}$ is the total number of back-propagation steps.

A.6 Ensemble Modeling

As shown in §5.2, the performance of BRIDGE on Spider is sensitive to the random seed. We train 10 different BRIDGE models with only differences in the random seeds. Figure A1 shows the performance of each individual model (sorted in decreasing exact set match), and the top- k models ensembled using average step probabilities.

The individual model performance variation is indeed large. The best and the worst models differ by 3.4 absolute points in E-SM, and 2.2 absolute points in execution accuracy.¹⁶ We hypothesize that this is a result of both intrinsic model variance

¹⁵https://fairseq.readthedocs.io/en/latest/lr_scheduler.html

¹⁶In general the execution accuracy of our model is lower than the E-SM. We believe the execution accuracy can be further improved by copying the anchor texts during SQL generation.

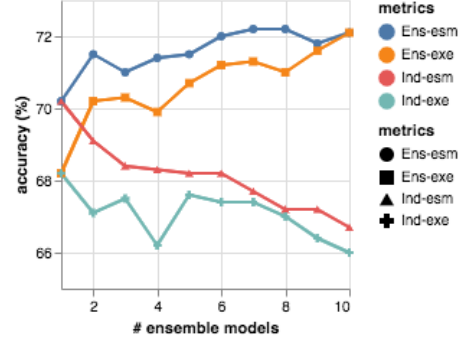


Figure A1: Performance ensemble models w.r.t. different # models in the ensemble.

	Best ✓	Best ✗
Worst ✓	61.2%	5.5%
Worst ✗	8.9%	24.4%

Table A4: Comparison of the best and worst model obtained via different random seeds in terms of error overlap on the Spider dev set.

as well as error in the evaluation metrics. Considering the false negatives, the true model performance could have less variance. Combining models in general leads to better performance. In particular, combining the best model with the second best model improves the E-SM by 1.3 absolute points. Further combining with the weaker models still shows improvements, but the return is diminishing. The top-7 model ensemble achieves the best E-SM (72.2%) and the top-10 model ensemble achieves the best execution accuracy (72.1%).

Table A4 shows the comparison between the best (70.2%) and worst (66.7%) models on the Spider dev set in terms of error overlap. For 61% of dev set, both models predicted the corrected answer and for 24.4% of dev set both models made a mistake. For 8.9% of the examples, only the best model is correct, while for 5.5% of the examples the worst model is correct. Manual examination shows that most of the examples where the two models evaluate differently are indeed different semantically.

A.7 Performance by Database

We further compute the E-SM accuracy of BRIDGE over different DBs in the Spider dev set. Figure A2 shows drastic performance differences across DBs. While BRIDGE achieves near perfect score on some, the performance is only 30%-40% on others. Performance does not always negatively correlate with the schema size. We hypothesize that the model scores better on DB schema similar

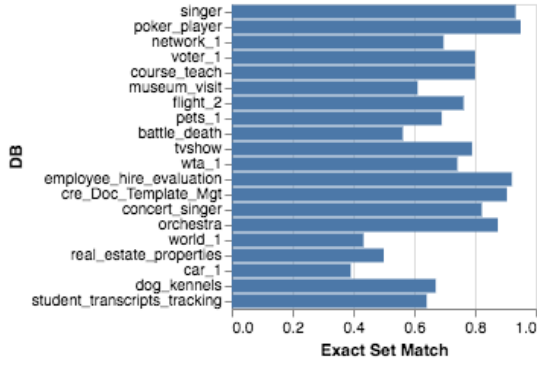


Figure A2: E-SM accuracy of BRIDGE by DB in Spider dev set. From top to bottom, the DBs are sorted by # tables in the schema in ascending order.

to those seen during training and better characterization of the “similarity” between DB schema could help transfer learning.

A.8 Visualizing fine-tuned BERT attention of BRIDGE

We visualize attention in the fine-tuned BERT layers of BRIDGE (with BERT-base-uncased) to qualitatively evaluate if the model functions as an effective text-DB encoder as we expect. We use the BERTViz library¹⁷ developed by Vig (2019).

We perform the analysis on the smallest DB in the Spider dev set to ensure the attention graphs are readable. The DB consists of two tables, `Poker_Player` and `People` that store information of poker players and their match results. While the BERT attention is a computation graph consisting of 12 layers and 12 heads, we were able to identify prominent patterns in a subset of the layers.

First, we examine if anchor texts indeed have the effect of bridging information across the textual and tabular segments. The example question we use is “show names of people whose nationality is not Russia” and “Russia” in the field `People.Nationality` is identified as the anchor text. As shown in Figure A3 and Figure A4, we find strong connection between the anchor text and their corresponding question mention in layer 2, 4, 5, 10 and 11.

We further notice that the layers effectively captures the relational DB structure. As shown in Figure A5 and Figure A6, we found attention patterns in layer 5 that connect tables with their primary keys and foreign key pairs.

We notice that all interpretable attention connections are between lexical items in the input se-

quence, not including the special tokens (`[T]`, `[C]`, `[V]`). This is somewhat counter-intuitive as the subsequent layers of BRIDGE use the special tokens to represent each schema component. We hence examined attention over the special tokens (Figure A7) and found that they function as bindings of tokens in the table names and field names. The pattern is especially visible in layer 1. As shown in Figure A7, each token in the table name “poker player” has high attention to the corresponding `[T]`. Similarly, each token in the field name “poker player ID” has high attention to the corresponding `[C]`. We hypothesize that this way the special tokens function similarly as the cell pooling layers proposed in TaBERT (Yin et al., 2020).

A.9 Future Improvements

We discuss a few aspects of BRIDGE that can be improved in future work.

Anchor Selection BRIDGE adopts simple string matching for anchor text selection. In our experiments, improving anchor text selection accuracy significantly improves the end-to-end accuracy. Extending anchor text matching to cases beyond simple string match (e.g. “LA”→“Los Angeles”) is a future direction. Furthermore, this step can be learned either independently or jointly with the text-to-SQL objective. Currently BRIDGE ignores number mentions. We may introduce features which indicate a specific number in the question falls within the value range of a specific column.

Input Size As BRIDGE serializes all inputs into a sequence with special tags, a fair concern is that the input would be too long for large relational DBs. We believe this can be addressed with recent architecture advancements in transformers (Beltagy et al., 2020), which have scaled up the attention mechanism to model very long sequences.

Relation Encoding BRIDGE fuses DB schema meta data features to each individual table field representations. This mechanism loses some information from the original graph structure. It works well on Spider, where the foreign key pairs often have exactly the same names. We consider regularizing a subset of the attention heads (Strubell et al., 2018) to capture DB connections a promising way to model the graph structure of relational DBs within the BRIDGE framework without introducing (a lot of) additional parameters.

¹⁷<https://github.com/jessevig/bertviz>

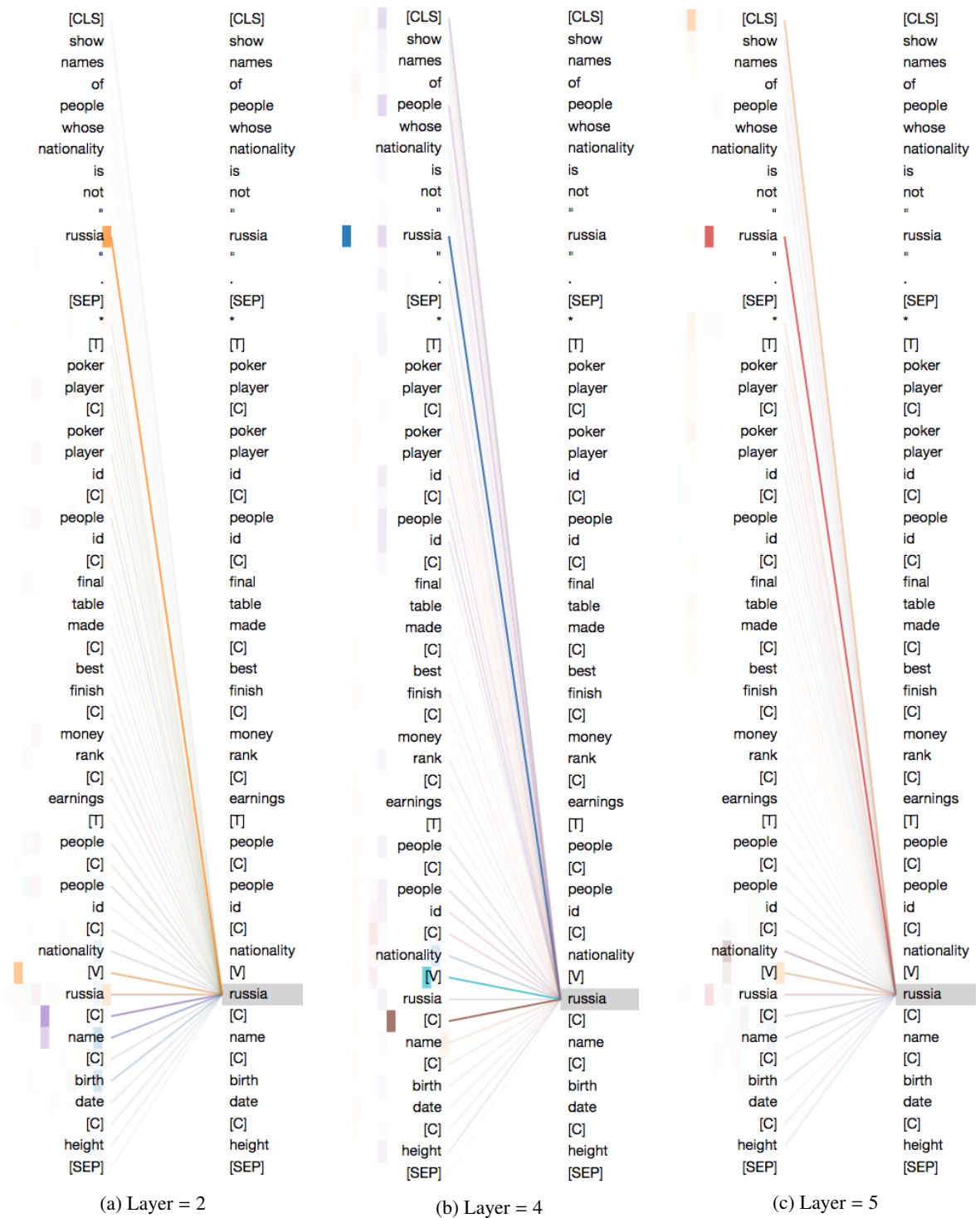


Figure A3: Visualization of attention to anchor text "Russia" from other words. In the shown layers, weights from the textual mention "Russia" is significantly higher than the other tokens.

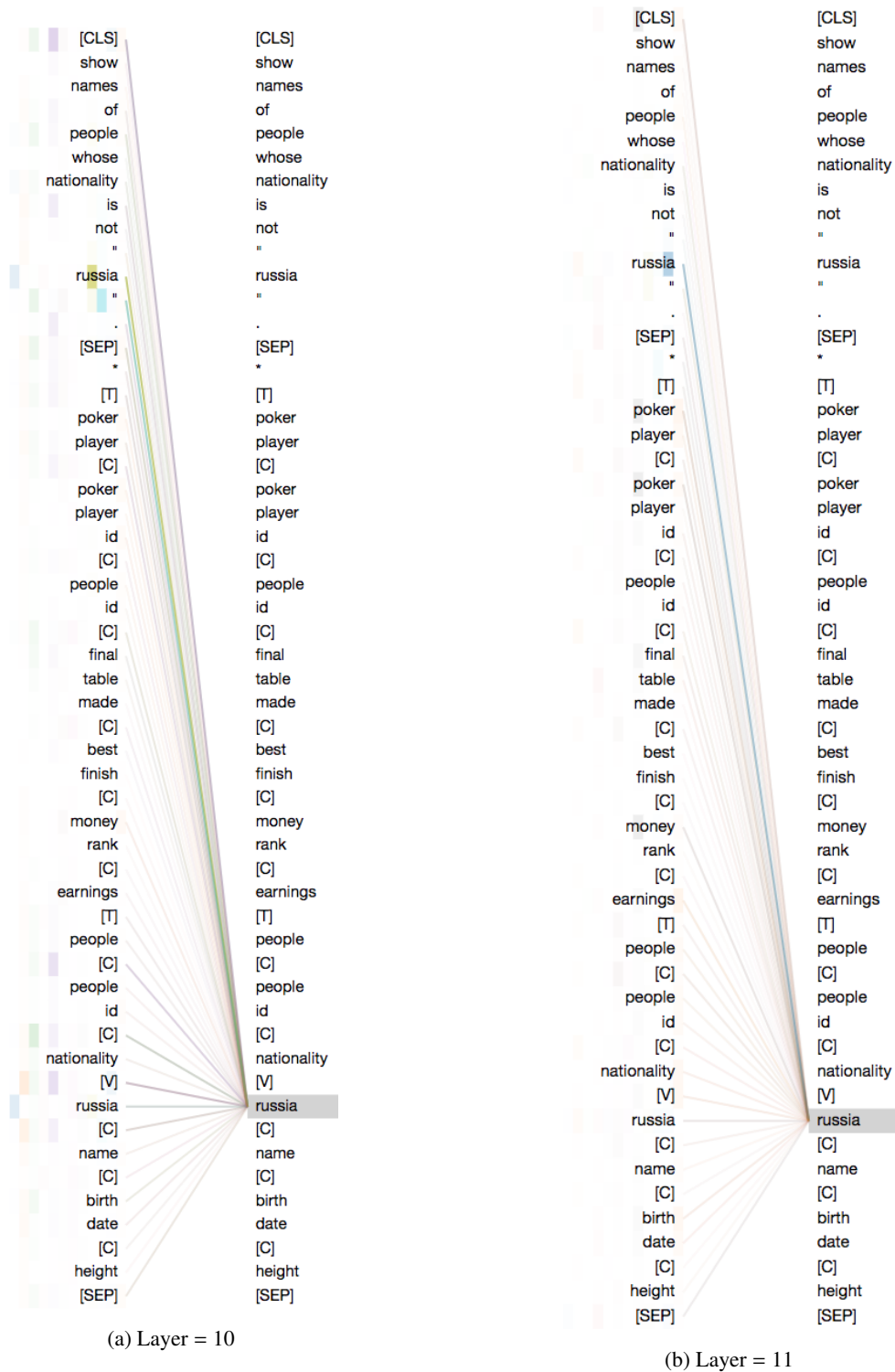


Figure A4: Visualization of attention to anchor text “Russia” from other words. Continue from Figure A3.

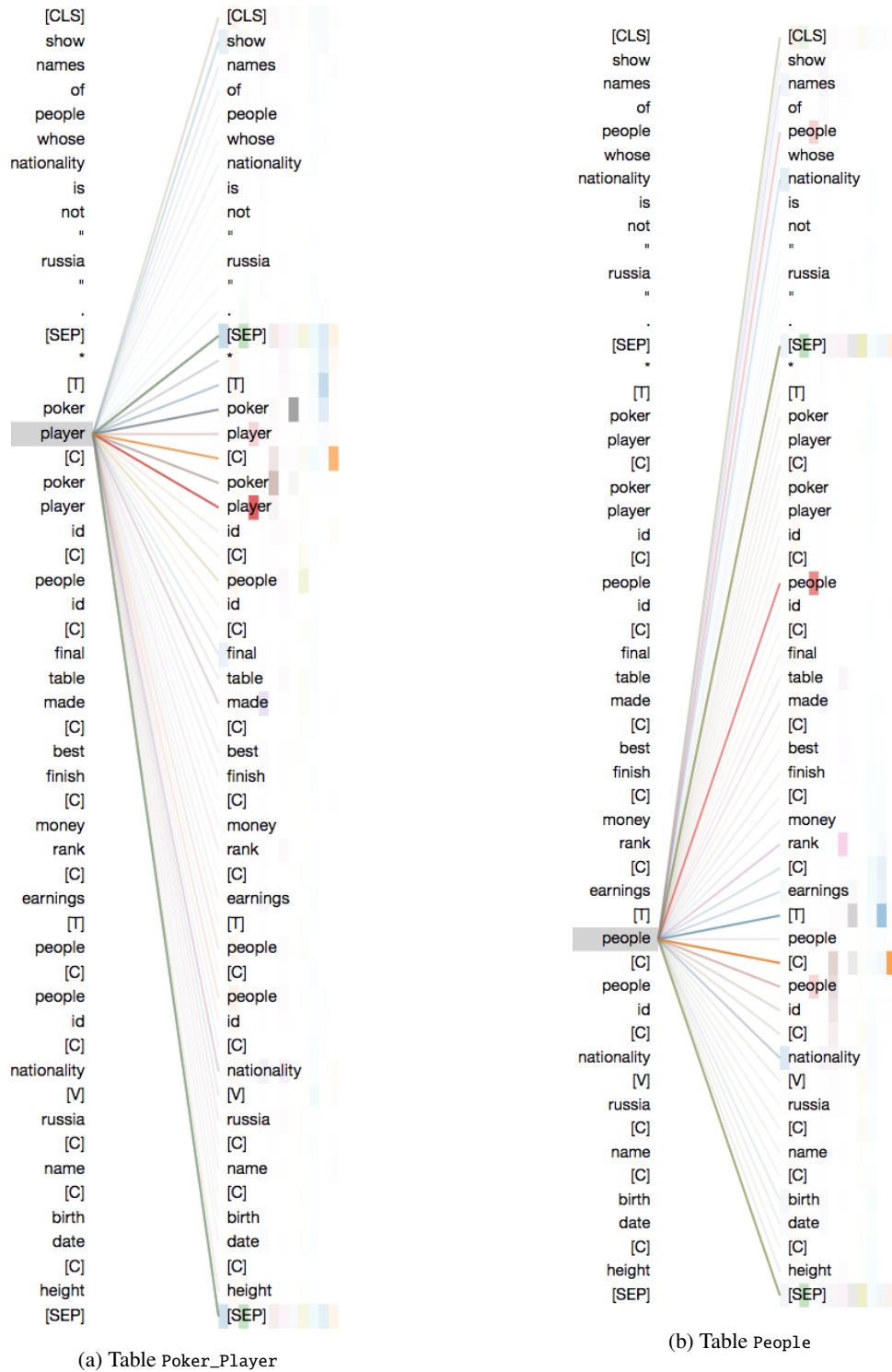
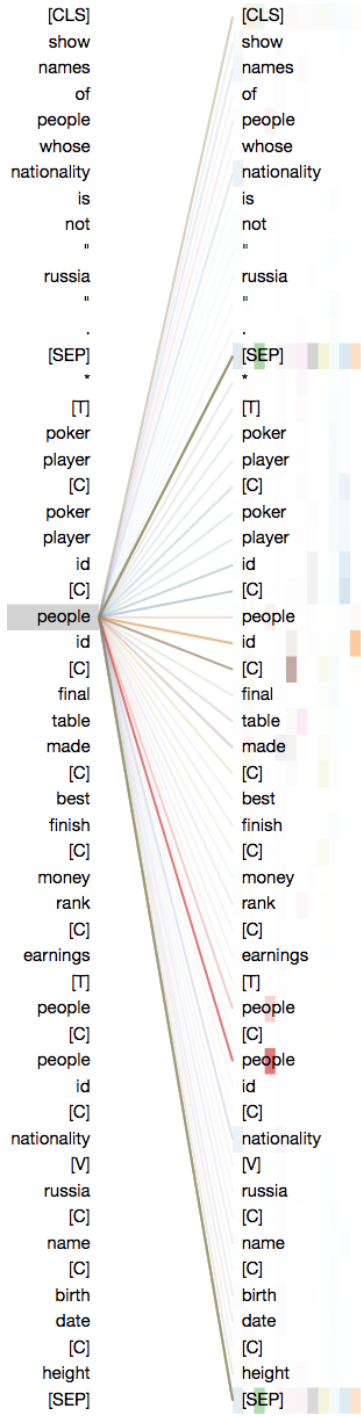
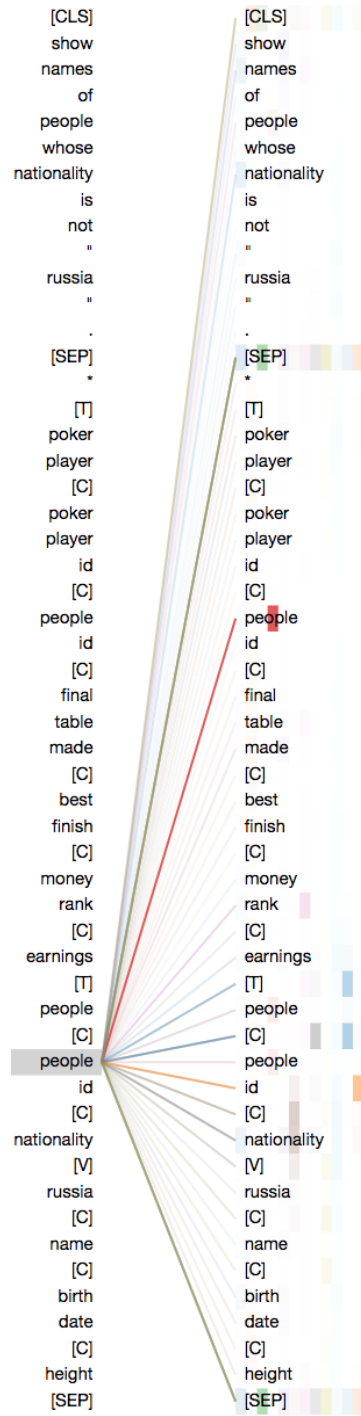


Figure A5: Visualization of attention in layer 5 from tables to their primary keys. In Figure A5b, the table name People has high attention weights to Poker_Player.People_ID, a foreign key referring to its primary key People.People_ID.



(a) Poker_Player.People_ID $\rightarrow$ People.People_ID



(b) People.People_ID $\rightarrow$ Poker_Player.People_ID

Figure A6: Visualization of attention in layer 5 between a pair of foreign keys.

